

# SDEV 2351

DAY 8: STATE PERSISTENCE PATTERNS

**SAVING FLUXOR STATE TO THE BROWSER**

## Session Objectives

1. Explain why SPA state is lost on page refresh and why persistence is needed
2. Compare localStorage and sessionStorage for different data types
3. Implement state persistence using `Blazored.LocalStorage` to save Fluxor state
4. Implement state rehydration to restore saved state on app startup

# Today's Schedule

| PHASE      | TIME   | WHAT'S HAPPENING                                             |
|------------|--------|--------------------------------------------------------------|
| Connecting | 10 min | The "refresh kills state" problem                            |
| Exploring  | 25 min | Persistence concepts + cart persistence demonstration        |
| Practicing | 55 min | Persist + rehydrate Recently Viewed + storage classification |
| Assessing  | 10 min | Exit ticket + Day 9 preview                                  |

PHASE 1 · 10 MINUTES

# CONNECTING

**Why did the cart disappear?**

# The Refresh Problem

Think back to Day 7's cart guard:

1. You added items to your cart
2. You navigated to checkout -- guard passed
3. You **refreshed the page**
4. Cart was empty -- guard redirected you to the shop

**Why?** Fluxor state lives in memory. Page refresh restarts the app. Memory is cleared. The store resets to initial state.

Today's question: **How do we make state survive a refresh?**

# Browser Storage Primer

Browser storage comes in two flavors:

- `localStorage` -- persists until you explicitly clear it
- `sessionStorage` -- cleared when the tab closes

Today we use `Blazored.LocalStorage`, a NuGet package that wraps browser `localStorage` with a clean C# API. No JavaScript needed.

PHASE 2 · 25 MINUTES

# EXPLORING

Persistence patterns with Blazored.LocalStorage

# localStorage vs. sessionStorage

| FEATURE  | LOCALSTORAGE                     | SESSIONSTORAGE             |
|----------|----------------------------------|----------------------------|
| Lifetime | Until explicitly cleared         | Until tab/window closes    |
| Scope    | Same origin, all tabs            | Same origin, single tab    |
| Size     | ~5 - 10 MB                       | ~5 - 10 MB                 |
| Use case | Preferences, cart, saved filters | One-time tokens, temp data |

**Security rule:** Never persist passwords or secrets in either storage. Both are accessible via JavaScript -- any XSS vulnerability exposes them.

# Decision Framework

| DATA                           | STORAGE        | WHY                                                         |
|--------------------------------|----------------|-------------------------------------------------------------|
| Shopping cart                  | localStorage   | User expects items to be there tomorrow                     |
| OIDC access token              | sessionStorage | Session - scoped, shorter lifespan                          |
| Dark mode preference           | localStorage   | Persistent, user expectation                                |
| Multi-step form progress       | sessionStorage | Start fresh next visit                                      |
| Passwords / API keys           | Never persist  | Damage if exposed via XSS                                   |
| "Show advanced options" toggle | Do not persist | Not sensitive — just nothing outside the component needs it |

# Blazored.LocalStorage API

The `ILocalStorageService` is registered via DI. Three key methods:

- `SetItemAsync<T>(key, value)` -- save (auto-serializes to JSON)
- `GetItemAsync<T>(key)` -- load (auto-deserializes from JSON)
- `RemoveItemAsync(key)` -- delete

# The Rehydration Pattern

**Hydrate** = refill an empty store from saved data. "Dry" empty object → load saved items back in → restored.  
(Same term as redux-persist / SSR hydration.)

Two complementary pieces:

**On state change:** An effect saves current state to localStorage after each reducer update

```
[EffectMethod]
public async Task HandleAddToCartAction(
    AddToCartAction action, IDispatcher dispatcher)
{
    await localStorage.SetItemAsync(
        "cart", cartState.Value.Items);
}
```

**On app startup:** Check localStorage for saved state, dispatch a hydration action, reducer restores the data

Full code: see [reference-sheet.md](#) or your IDE

PHASE 3 · 55 MINUTES

# PRACTICING

**Persist, rehydrate, and classify**

# Gradual Release

I-DO → WE-DO → YOU-DO

- **I-do** — you watched me build the full persistence + hydration pattern in the Demonstration, **on the cart**.
- **Worked launch** — I add the **save effect** for Recently Viewed on screen as one worked example.
- **You-do** — you build the rest of the layer on your own: the two hydration actions, the reducer, the hydration effect, and the `MainLayout` wiring — then the classification activity.

This is **transfer, not reproduction**. Practice 1 is a different feature from the demonstration on purpose.

## Practice 1: Persist + Rehydrate "Recently Viewed" (35 min)

WORKED LAUNCH → INDEPENDENT

**A different feature from the demonstration.** You watched the whole pattern on the cart; here you build the **whole** persistence + hydration layer yourself, on a feature you have not wired.

**Starter provides (all working, in memory):** `RecentlyViewed` state, `ViewProductAction` with its prepend/dedupe/cap-at-5 reducer, the Shop "View" buttons, the display list and the navbar count. Blazored.LocalStorage is installed and registered. `RecentlyViewedEffects.cs` is a shell — DI and the storage-key constant only. It loses the list on refresh.

**Worked launch — the save effect.** I add the `[EffectMethod]` for `ViewProductAction` on screen:  
`localStorage.SetItemAsync(RecentlyViewedStorageKey, recentState.Value.Items)`.

**Effect Timing Rule:** save the list **from state**, not `action.Item` — the reducer has already run.

## Practice 1 (cont.) — the independent remainder

Then you transfer the rest of the pattern on your own:

`RecentlyViewedActions.cs` — two hydration actions

- `HydrateRecentlyViewedRequestAction`
- `HydrateRecentlyViewedAction(IReadOnlyList<RecentlyViewedItem> Items)`

`RecentlyViewedReducers.cs` — the matching `[ReducerMethod]`

- `state with { Items = action.Items }`

`RecentlyViewedEffects.cs` — `HandleHydrateRecentlyViewedRequestAction`

- Read with `GetItemAsync<List<RecentlyViewedItem>>`, dispatch `HydrateRecentlyViewedAction` if items are found

`MainLayout.razor` — `OnAfterRenderAsync(firstRender)` dispatches `HydrateRecentlyViewedRequestAction`

# Storage Classification (15 min)

PAIR WORK — NO CODE

No coding. For each scenario, decide: **localStorage**, **sessionStorage**, or **do not persist**. Justify your choice.

| # | SCENARIO                       | YOUR DECISION |
|---|--------------------------------|---------------|
| 1 | Shopping cart items            | ?             |
| 2 | User's preferred language      | ?             |
| 3 | OAuth access token             | ?             |
| 4 | "Show advanced options" toggle | ?             |
| 5 | Search filter settings         | ?             |
| 6 | Credit card number             | ?             |
| 7 | Multi-step form progress       | ?             |
| 8 | Dark/light theme preference    | ?             |

# Troubleshooting

| PROBLEM               | CAUSE                                                 | FIX                                                                                  |
|-----------------------|-------------------------------------------------------|--------------------------------------------------------------------------------------|
| State not saving      | Calling <code>SetItemAsync</code> before reducer runs | Effects run after reducers - - read <code>IState&lt;T&gt;.Value</code> in the effect |
| Null on first load    | No data in localStorage yet                           | Check for null before dispatching hydration action                                   |
| Deserialization error | Data shape changed since last save                    | Wrap <code>GetItemAsync</code> in try/catch, fallback to empty state                 |

PHASE 4 · 10 MINUTES

# ASSESSING

# Summary

Today you learned to:

1. **Explain why SPA state is lost on refresh** -- Fluxor state lives in memory, refresh resets it
2. **Compare localStorage and sessionStorage** -- lifetime, scope, and appropriate use cases
3. **Implement state persistence** using `Blazored.LocalStorage` to save Fluxor state after changes
4. **Implement state rehydration** to restore saved state on app startup with error handling

# Exit Ticket

In one sentence:

Why does refreshing the browser clear your Fluxor state? Name one thing that would break in our app without persistence.

# What's Next

## Day 9: Advanced Routing Patterns + Lab 1 Assignment

What happens when someone types a URL that doesn't exist? We'll cover **fallback routes**, **catch-all parameters**, and **route precedence**.

Then **Lab 1 drops** -- the last 30 minutes of Day 9 is the lab assignment walkthrough.